

Lecture IV - Handling Data in more than one Dimension

Programming with Python

Dr. Tobias Vlček

Kühne Logistics University Hamburg - Fall 2026

EP 4: Menu Grows Up

Seventeen variables and counting

The menu started with two dishes. Tobi tracked them in variables: `price1`, `price2`. Then came `price3`, a `price_final`, and, after a rough Tuesday, a `price_final_FINAL2`.

...

Seventeen variables for one menu. Add a dish and you touch seventeen lines. Today the data gets **structure**: one name that holds many values, and a way to look things up by name instead of by counting.

Warm-up

Question 1

Three questions from Episode 3.

```
def announce(item):  
    print(f"Fresh today: {item}")  
  
result = announce("Mate")  
print(result)
```

The **second** `print` shows what?

a) `Fresh today: Mate` b) `Error` c) `None`

Answer 1

```
def announce(item):  
    print(f"Fresh today: {item}")  
  
result = announce("Mate")  
print(result)
```

```
Fresh today: Mate  
None
```

...

c) **None**: `announce` has no `return`, so it hands back `None`. Printing something is not the same as returning it.

Question 2

```
def restock(n):  
    n = n + 10  
    return n  
  
stock = 5  
restock(stock)  
print(stock)
```

a) 5 b) 15 c) Error

Answer 2

```
def restock(n):  
    n = n + 10  
    return n  
  
stock = 5  
restock(stock)  
print(stock)
```

5

...

a) 5. `n = n + 10` points the function's own name `n` at a new number, so the global `stock` never moves. We never caught the result. To keep a result, assign it back.

Question 3

```
class Order:  
    def __init__(self, item, qty, price):  
        self.item = item  
        self.qty = qty  
        self.price = price  
  
    def total(self):  
        return self.qty * self.price  
  
print(Order("Wrap", 2, 6.90).total())
```

a) 6.9 b) 13.8 c) 2

Answer 3

```
class Order:
    def __init__(self, item, qty, price):
        self.item = item
        self.qty = qty
        self.price = price

    def total(self):
        return self.qty * self.price

print(Order("Wrap", 2, 6.90).total())
```

13.8

...

b) 13.8. `total()` multiplies `qty` by `price`: `2 * 6.90`, and `print` shows a float without the trailing zero. The object carries its own data, the method does the arithmetic.

Lists & Tuples

One name for many values

A **list** holds many values under one name, in order. That's the cure for seventeen variables:

```
drinks = ["Mate", "Spezi", "Ayran"]
print(drinks)
```

['Mate', 'Spezi', 'Ayran']

...

Square brackets `[]`, items separated by commas. Add a drink tomorrow and the list just grows. No new variable names.

Reaching in by position

Each item has an **index**. Python counts from **0**:

```
drinks = ["Mate", "Spezi", "Ayran"]
print(drinks[0])    # the first item
print(drinks[2])    # the third item
```

Mate
Ayran

...

The first item is `drinks[0]`, not `drinks[1]`. Off-by-one is a classic mistake of students.

Counting from the end

A **negative** index counts backwards from the end, no need to know the length:

```
drinks = ["Mate", "Spezi", "Ayran"]
print(drinks[-1])    # last item
print(drinks[-2])    # second to last
```

```
Ayran
Spezi
```

...

`-1` is always the last item. Useful for getting “the most recent order”.

Slicing: a range of items

A **slice** `list[start:stop]` returns a new list, from `start` up to, but **not including**, `stop`:

```
drinks = ["Mate", "Spezi", "Ayran"]
print(drinks[0:2])    # items 0 and 1
print(drinks[-2:])    # the last two: negative start, open end
```

```
['Mate', 'Spezi']
['Spezi', 'Ayran']
```

...

The `stop` index is left out. `drinks[0:2]` gives you two items, not three. And leaving a side **empty** means “all the way”: `drinks[-2:]` starts two from the end and runs to the finish.

Predict: where does the slice stop?

A courier carries four cup sizes. What does this print?

```
sizes = ["S", "M", "L", "XL"]
print(sizes[1:3])
```

a) `['M', 'L', 'XL']` b) `['M', 'L']` c) `['S', 'M', 'L']`

...

Predict. Pick a letter, then I reveal the answer.

Answer: the `stop` index is excluded

```
sizes = ["S", "M", "L", "XL"]
print(sizes[1:3])
```

```
['M', 'L']
```

...

b) `['M', 'L']`. The slice starts at index 1 (`"M"`) and stops before index 3, so index 3 (`"XL"`) is never included. A slice from `1:3` gives you exactly $3 - 1 = 2$ items.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_04_a/



Any questions? Let me know!

Growing a list

Lists are **mutable**: you can change them after creation. `.append()` adds one item to the end; `+` joins two lists into a new one:

```
drinks = ["Mate", "Spezi"]
drinks.append("Ayrar")
print(drinks)

combined = ["Mate"] + ["Kombucha"]
print(combined)
```

```
['Mate', 'Spezi', 'Ayrar']
['Mate', 'Kombucha']
```

...

`.append()` changes the list itself, and every name pointing at it sees that; `+` builds a fresh one. `=` never changes a list: it only points a name at one.

Predict: the second name

Tobi wants a winter copy of the drinks list to experiment on. What does the last line print?

```
summer = ["Mate", "Spezi"]
winter = summer
winter.append("Ayrán")
print(summer)
```

a) ['Mate', 'Spezi'] b) ['Ayrán', 'Mate', 'Spezi'] c) ['Mate', 'Spezi', 'Ayrán']

...

Predict. Pick a letter, then I reveal the answer.

Answer: two names, one list

```
summer = ["Mate", "Spezi"]
winter = summer
winter.append("Ayrán")
print(summer)
```

```
['Mate', 'Spezi', 'Ayrán']
```

...

c) ['Mate', 'Spezi', 'Ayrán']. `winter = summer` does not copy anything: both names point at the **same** list, so `.append()` through one name shows up under the other. A real copy is `list(summer)` or `summer[:]` (a slice with both ends open, so all of it).

How long is it?

`len()` tells you how many items a list holds:

```
drinks = ["Mate", "Spezi", "Ayrán"]
print(len(drinks))
```

```
3
```

...

`len()` also works on strings (letters) and, soon, on dictionaries (keys).

Looping over positions

A loop over the list hands you each item. Sometimes you need to know **where** an item sits. Then loop over the positions instead:

```
drinks = ["Mate", "Spezi", "Ayran"]
for i in range(len(drinks)):
    print(i, drinks[i])
```

```
0 Mate
1 Spezi
2 Ayran
```

...

`range(len(drinks))` counts 0, 1, 2: one number per position, and `drinks[i]` is the item that sits there.

Tuples: fixed-length records

A **tuple** looks like a list but uses `()` and **cannot be changed**, perfect for a record whose shape never varies, like opening hours `(hour, minute)`:

```
opening = (9, 0)    # 9:00 sharp
print(opening[0])
print(opening[1])

hour, minute = opening    # unpack: one name per slot
print(hour, minute)
```

```
9
0
9 0
```

...

i Note

Indexing and slicing work exactly as on lists. You just can't `.append()` to a tuple.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_04_b/



Any questions? Let me know!

Dictionaries & Sets

Look things up by name

A list finds items by position. But “what does a Spezi cost?” is a question about a name, not a position. A **dictionary** maps a key to a value:

```
prices = {"Mate": 3.50, "Spezi": 3.20, "Ayrar": 2.80}
print(prices["Spezi"])
```

```
3.2
```

...

Curly braces {}, each pair written **key: value**. No more remembering that Spezi lives at index 1.

Building a dictionary

Keys are usually strings; values can be anything. Read a value with square brackets and its key:

```
prices = {"Mate": 3.50, "Spezi": 3.20, "Ayrar": 2.80}
print(prices["Mate"])
print(prices["Ayrar"])
```

```
3.5
2.8
```

...

The same square brackets as a list, but now the thing inside is a **key**, not a number.

The key that isn't there

Ask for a key that doesn't exist and Python raises a **KeyError**. It stops rather than guess:

```
prices = {"Mate": 3.50, "Spezi": 3.20}
print(prices["Cola"])
# KeyError: 'Cola'
```

...

⚠ Warning

A **KeyError** means the key is spelled wrong or was never added. However, there is a smarter way to ask for a key.

A safer read with **.get()**

.get() returns **None** for a missing key instead of raising, and you can supply a fallback:

```
prices = {"Mate": 3.50, "Spezi": 3.20}
print(prices.get("Cola"))           # missing → None
print(prices.get("Cola", 0))        # missing → your fallback
```

```
None
0
```

...

Use **[]** when the key **must** be there; use **.get()** when it might not.

Predict: is it on the menu?

in asks whether a value is in a list: **"Mate" in drinks** is **True**. On a dictionary, what does it look at?

```
prices = {"Mate": 3.50, "Spezi": 3.20}
print("Spezi" in prices, 3.20 in prices)
```

a) True False b) True True c) False True

...

Predict. Pick a letter, then I reveal the answer.

Answer: **in** checks the keys

```
prices = {"Mate": 3.50, "Spezi": 3.20}
print("Spezi" in prices, 3.20 in prices)
print(3.20 in prices.values())
```

```
True False
True
```

...

a) **True False**. Membership on a dictionary looks at the **keys** only. `3.20` is a value, so Python says it is not there. To search the values, ask `3.20 in prices.values()`.

Updating and adding

Assigning to a key updates it if it exists, or adds it if it doesn't. Work on a **copy** (`dict()`) when the original must survive:

```
prices = {"Mate": 3.50, "Spezi": 3.20}
winter = dict(prices)           # a copy: keep the original safe
winter["Spezi"] = 3.40          # update an existing key
winter["Kombucha"] = 4.20       # add a brand-new key
print(winter)
print(prices)                  # untouched (the summer menu comes back in April)
```

```
{'Mate': 3.5, 'Spezi': 3.4, 'Kombucha': 4.2}
{'Mate': 3.5, 'Spezi': 3.2}
```

...

One syntax, two jobs. Python decides by whether the key is already present.

Predict: the missing drink

A customer orders a Cola. It's not on the menu. What does this line do?

```
prices = {"Mate": 3.50, "Spezi": 3.20}
print(prices["Cola"])
```

a) prints `None` b) prints `""` c) raises `KeyError` d) adds `"Cola"`

...

Predict. Pick a letter, then I reveal the answer.

Answer: reading with `[]`

```
prices = {"Mate": 3.50, "Spezi": 3.20}
print(prices["Cola"])
```

```
KeyError: 'Cola'
Traceback (most recent call last):
  File "script.py", line 1, in <module>
    print(prices["Cola"])
KeyError: 'Cola'
```

...

c) raises **KeyError**: square-bracket reads never invent a value and never add one. Reach for `.get()` when a key might be missing.

Sets: only the unique ones

A **set** keeps each value **once**. Feed it duplicates and they collapse. Perfect for counting *distinct* things, like today's regulars:

```
visitors = ["nina", "tom", "nina", "ada", "tom"]
regulars = set(visitors)
print(regulars)          # order is arbitrary: a set has none
print(len(regulars))     # how many different people
```

```
{'ada', 'tom', 'nina'}
3
```

...

Five visits, three people. A set answers “how many *different*?”.

...

Can be created from a list, or with curly braces:

```
regulars = {"nina", "tom", "ada"}
print(regulars)
```

```
{'ada', 'tom', 'nina'}
```

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_04_c/



Any questions? Let me know!

Nested Data

A dictionary inside a dictionary

Values can themselves be dictionaries. Each delivery **zone** carries its own fee and estimated time:

```
zones = {  
    "Altstadt": {"fee": 1.50, "eta": 12},  
    "Hafen": {"fee": 2.50, "eta": 20},  
}  
print(zones["Hafen"])
```

```
{'fee': 2.5, 'eta': 20}
```

...

One key ("Hafen") points to a whole dictionary. This is how real data grows: structures inside structures.

Reading nested values

To reach an inner value, use **two** keys in a row, outer first, then inner:

```
zones = {  
    "Altstadt": {"fee": 1.50, "eta": 12},  
    "Hafen": {"fee": 2.50, "eta": 20},  
}  
print(zones["Hafen"]["fee"])  
print(zones["Altstadt"]["eta"])
```

```
2.5
12
```

...

Read it left to right: “in **zones**, take **Hafen**, then its **fee**.”

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_04_d/



Any questions? Let me know!

From a loop to one line

You already know the loop that builds a list. A **list comprehension** is that same loop, written on one line:

```
counts = [2, 1, 3]

# the loop you know
doubled = []
for c in counts:
    doubled.append(c * 2)
print(doubled)

# the one-liner
doubled = [c * 2 for c in counts]
print(doubled)
```

```
[4, 2, 6]
[4, 2, 6]
```

...

Read it as “`c * 2` for each `c` in `counts`”. Same result, less typing.

round inside a comprehension

Any expression can go in front of the `for`, including `round()`. Loyalty pricing: 10% off, cleaned to two decimals:

```
drink_prices = [3.50, 3.20, 2.80]

print([p * 0.9 for p in drink_prices])      # raw floats, ugly
print([round(p * 0.9, 2) for p in drink_prices]) # rounded, clean
```

```
[3.15, 2.8800000000000003, 2.52]
[3.15, 2.88, 2.52]
```

...

The raw version leaks a `2.88000...3` float. Wrapping each price in `round(_, 2)` is the same thing we already used for money.

Looping over a dictionary

`.items()` hands you the menu as `(key, value)` pairs. Give the `for` two names and each pair is unpacked into them, like `hour, minute = opening`:

```
prices = {"Mate": 3.50, "Spezi": 3.20, "Ayran": 2.80}
for name, price in prices.items():
    print(name, price)
```

```
Mate 3.5
Spezi 3.2
Ayran 2.8
```

...

One name per slot, left to right. Two names, two slots.

Comprehensions build dictionaries

Swap the brackets for braces and give a `key: value`. Now you rebuild a whole menu in one line:

```
prices = {"Mate": 3.50, "Spezi": 3.20, "Ayran": 2.80}
loyalty = {name: round(price * 0.9, 2) for name, price in prices.items()}
print(loyalty)
```

```
{'Mate': 3.15, 'Spezi': 2.88, 'Ayran': 2.52}
```

...

The same `name, price` loop, now inside braces. The whole discounted menu, no loop body in sight.

Predict: the comprehension

What does this build?

```
nums = [1, 2, 3, 4]
print([n * n for n in nums])
```

a) [1, 4, 9, 16] b) [2, 4, 6, 8] c) [1, 2, 3, 4]

...

Predict. Pick a letter, then I reveal the answer.

Answer: square each item

```
nums = [1, 2, 3, 4]
print([n * n for n in nums])
```

```
[1, 4, 9, 16]
```

...

a) [1, 4, 9, 16]. The expression `n * n` runs once per item, so each number becomes its own square. `2 * n` would double them; the bare `n` would just copy the list.

Predict: comprehension with filter

Some dishes sold nothing today. A comprehension can carry an `if` after the `for`. How long is the result?

```
counts = [2, 0, 3, 0, 1]
print([c * 2 for c in counts if c > 0])
```

a) [4, 0, 6, 0, 2] b) [4, 6, 2] c) [2, 3, 1]

...

Predict. Pick a letter, then I reveal the answer.

Answer: the `if` drops items

```
counts = [2, 0, 3, 0, 1]
print([c * 2 for c in counts if c > 0])
```

```
[4, 6, 2]
```

...

b) `[4, 6, 2]`. The `if` is a filter: items that fail it never reach the expression, so the result is **shorter** than the input. `c * 2` still runs on the survivors, which rules out `[2, 3, 1]`.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_04_e/



Any questions? Let me know!

To the Lab

After the break: the lab

- Head to the lab notebook: [Episode 4: The Menu Grows Up](#)
- You'll rebuild the menu as a dictionary, count the regulars with a set, discount the card with a comprehension, and steer a lost courier through a **nested campus map** to the dorms
- It runs entirely in your browser: no setup, just click and code

...

! Important

Download your `.py` before you leave. Closing the tab without downloading loses your work, and downloading is exactly how you hand in the checkpoints.

Wrap-up

Three things to remember

1. **Lists** hold many values in order: index from `0`, count back with `-1`, slice with a `stop` that's **excluded**
2. **Dictionaries** look up a value by its key; a missing key raises `KeyError`, so reach for `.get()` when it might not be there; **sets** keep each value once
3. **Comprehensions** compress a loop into one line: `[expr for x in things]` for lists, `{k: v for ...}` for dicts, `round()` welcome inside

...

Note

Next time, Episode 5 starts with **Checkpoint 2**: 40 minutes, no AI, everything from Sessions III-IV. Then Tobi rewrites the checkout at **3 AM** on four energy drinks. What could go wrong? We learn to read tracebacks and catch errors before the customer does.

Literature

Books to start with

- Downey, A. B. (2024). Think Python: How to think like a computer scientist (Third edition). O'Reilly. [Link to free online version](#)
- Elter, S. (2021). Schrödinger programmiert Python: Das etwas andere Fachbuch (1. Auflage). Rheinwerk Verlag.

...

Note

Nothing new here, but these are still great books to start with!

...

For more, see the [literature list](#) of this course.